

RansomShield — Discours de soutenance

10 min de présentation + 5 min de démo

Codjo Fréjus Loïc SANGRONIO — 24 juin 2026

Lire à voix haute avant la soutenance. Les **flèches vertes** sont des phrases à prononcer, pas à lire : elles enchainent les diapos naturellement. Les **mots en gras** sont les ancres à mémoriser en priorité.

Diapo 1/25 — OUVERTURE — 0 :00

Monsieur le Président du jury, Mesdames et Messieurs les membres du jury, bonjour.

Je m'appelle Loïc SANGRONIO, et je vais vous présenter aujourd'hui **RansomShield** : un système de détection et de réponse aux attaques ransomware, dans un environnement contrôlé.

Dix minutes de présentation. Cinq minutes de démonstration live, sur des machines virtuelles réelles. Et je serai à votre disposition pour vos questions à la fin.

⇒ **Commençons par un coup d'œil au plan.**

Diapo 2/25 — SOMMAIRE — 0 :30

Cette présentation suit un fil simple : d'abord **le pourquoi** — le contexte et les objectifs ; ensuite **le quoi** — la conception et les choix techniques ; enfin **le résultat** — les tests, les limites, et la démonstration.

⇒ **Commençons par le contexte.**

Diapo 4/25 — CONTEXTE & PROBLÉMATIQUE — 1 :00

Imaginez : vos fichiers se renomment les uns après les autres, les sauvegardes disparaissent, et un message s'affiche à l'écran — « payez, ou perdez tout ». C'est un ransomware.

L'ENISA a mesuré une **hausse de 73 % de ces attaques en 2023**. Sans outil dédié, il faut en moyenne **21 jours pour détecter** l'incident, pour un coût qui dépasse souvent le million de dollars.

Le problème fondamental ? Les antivirus classiques détectent par signature. Un ransomware inconnu passe à travers.

D'où la question centrale de ce travail : **comment détecter un comportement ransomware, dans un environnement contrôlé, sans déployer de vrai malware ?**

⇒ **C'est précisément ce que RansomShield cherche à faire — voyons les objectifs.**

Diapo 5/25 — OBJECTIFS — 2 :30

Six objectifs concrets : surveiller les événements **en temps réel**, calculer un **score de risque**, générer automatiquement des alertes et des incidents, proposer des actions de protection — avec un **contrôle humain à chaque étape** —, sécuriser la console par une **double authentification**, et valider l'ensemble sur des **scénarios d'attaque réalistes**.

⇒ **Pour y répondre, le mémoire s'articule en trois chapitres.**

Diapo 6/25 — ORGANISATION DU MÉMOIRE — 3 :15

Revue de littérature. Conception du système. Réalisation et résultats. Voyons d'abord les fondements théoriques.

⇒ **Qu'est-ce qu'un ransomware, exactement ?**

Diapo 8/25 — DÉFINITION & COMPORTEMENTS — 3 :50

Le NIST définit le ransomware comme un logiciel malveillant qui **bloque l'accès aux données** — le plus souvent par chiffrement — pour ensuite réclamer une rançon.

Ce qui nous intéresse ici : ces attaques **ne sont pas silencieuses**. Elles laissent des traces avant que les dégâts ne soient irréversibles : des fichiers renommés en `.locked`, une note de rançon qui apparaît, les sauvegardes qui s'effacent, ou des outils système détournés de leur usage normal.

Ce sont ces signaux comportementaux qui constituent notre détection.

⇒ **Mais pourquoi les outils existants ne suffisent-ils pas ?**

Diapo 9/25 — SOLUTIONS EXISTANTES — 5 :15

Les antivirus ? Signatures connues ; inefficaces sur les variantes nouvelles.

Les SIEM ? Ils collectent des journaux — sans corrélation comportementale ciblée.

Les EDR commerciaux ? CrowdStrike, SentinelOne sont efficaces — et hors de portée pour un contexte académique.

RansomShield se positionne dans cet espace vide : approche comportementale, open-source, avec un opérateur humain dans la boucle à chaque décision.

⇒ **Concrètement, comment est-il construit ?**

Diapo 11/25 — ARCHITECTURE — 6 :15

Le flux est simple à suivre. Les **agents Python** surveillent les machines et envoient les événements à l'**API Laravel**. Le **moteur de détection** analyse, calcule un score, et déclenche une alerte si le seuil est dépassé. L'opérateur voit tout sur la **console SOC** et **valide chaque action avant son exécution**.

Règle d'or du système : **aucune action automatique sans validation humaine**.

⇒ **Ce flux repose sur quatre composants — regardons-les.**

Diapo 12/25 — LES 4 COMPOSANTS — 7 :15

1. **L'agent Python** : surveille les fichiers en temps réel, conserve une file SQLite locale pour ne rien perdre en cas de coupure réseau.

2. **La console SOC** : centralise alertes, incidents, et file d'approbation.

3. **La découverte réseau** : scanne les sous-réseaux automatiquement, mais aucun agent ne s'inscrit sans validation manuelle de l'administrateur.

4. **Le module de simulation** : rejoue jusqu'à 22 événements d'attaque — sans jamais toucher à un vrai malware.

⇒ **Deux acteurs utilisent ce système — qui sont-ils ?**

Diapo 13/25 — DIAGRAMME UML — 8 :45

Deux acteurs, deux rôles distincts.

L'**administrateur** : il configure le système, gère les agents, et lance les simulations.

L'**analyste SOC** : il traite l'opérationnel — il visualise les alertes, approuve ou rejette les actions, consulte la timeline d'audit. Mais il n'a pas accès à la configuration.

⇒ Ces acteurs manipulent des données — voyons comment elles sont structurées.

Diapo 14/25 — MODÈLE DE DONNÉES — 9 :45

Huit tables MySQL. Le pipeline se lit de gauche à droite : **agent** → **événement** → **alerte** → **incident** → **action** → **journal d'audit**.

Chaque action est signée : on sait **qui** l'a validée, et **quand**. Aucune décision n'est anonyme.

⇒ Passons maintenant à la réalisation concrète.

Diapo 16/25 — CHOIX TECHNIQUES — 11 :00

Backend : **Laravel 11** et PHP 8.3, base **MySQL**, API sécurisée par une clé propre à chaque agent.

Agent : **Python** + **Watchdog**, file locale **SQLite**.

Frontend : **Chart.js** en local, sans dépendance externe.

Infrastructure de test : **3 VMs KVM**, double authentification activée.

⇒ Concrètement, comment l'agent fonctionne-t-il au démarrage ?

Diapo 17/25 — AGENT & MOTEUR — 12 :15

Au premier démarrage, l'agent s'enrôle via un **token à usage unique** — il obtient une clé API permanente. Ensuite : surveillance des dossiers avec Watchdog, et **heartbeat toutes les 30 secondes**.

Le moteur calcule un score cumulatif. Exemple réel : fichier renommé **.locked = 80 points**, note de rançon détectée = **55 points**, total **135** → **alerte critique** + **incident créé automatiquement**.
Durée totale : **moins de 5 secondes**.

⇒ Côté analyste, qu'est-ce que la console offre ?

Diapo 18/25 — CONSOLE SOC — 13 :30

Dix pages fonctionnelles. Le dashboard se rafraîchit **toutes les 30 secondes** automatiquement.

Chaque action de protection passe par une **file d'approbation** : l'analyste valide ou rejette avant toute exécution. Les notifications partent sur **quatre canaux simultanés** : web, e-mail, son, et webhook.

Vous allez voir tout ça dans quelques instants.

⇒ Mais d'abord — qu'est-ce que les tests ont donné ?

Diapo 19/25 — TESTS & RÉSULTATS — 14 :45

Cinq scénarios d'attaque, de 7 à 22 événements. **Détection en moins de 5 secondes** à chaque fois. Zéro perte d'événement. Pipeline validé **de bout en bout** : de l'événement jusqu'à la timeline d'audit.

Une limite que j'assume honnêtement : les tests ont été conduits **uniquement sur Linux**. Un attaquant qui ralentirait délibérément son rythme resterait moins visible.

⇒ **Maintenant, place à la démonstration.**

Diapo 20/25 — DÉMO LIVE — 15 :00 (checklist — 5 min, hors chrono)

1. Ouvrir `http://10.20.0.1` — console SOC
2. Se connecter : identifiants + **code 2FA**
3. Vérifier : **3 agents en ligne** (heartbeat vert)
4. **Agents** → VM-02 → **Lancer simulation** → « Kill chain complète » (22 évts)
5. **Dashboard** : le score monte en temps réel
6. **Alertes** : niveau Critique + signaux détectés
7. **Incidents** : ouvrir l'incident auto-crée
8. **File d'approbation** : approuver l'isolation réseau
9. **Timeline** : tout est horodaté et tracé

Rester calme ; chaque étape a le temps qu'il lui faut.

⇒ **Revenons à la présentation pour conclure.**

RansomShield couvre **tout le cycle de traitement d'un incident** : collecte, analyse, alerte, réponse, audit. Agent multi-plateforme. Console complète. Moteur réactif. **Contrôle humain systématique**.

Ce travail a ses limites : environnement contrôlé, réponses partiellement simulées, tests conduits sur Linux uniquement.

Les perspectives sont claires : d'abord le **chiffrement HTTPS**, puis des rôles utilisateurs distincts, puis à terme du **machine learning** pour affiner la détection, et une **intégration avec les SIEM** du marché.

Je vous remercie pour votre attention. **Je suis à votre disposition pour vos questions.**

Diapo 23/25 — QUESTIONS DU JURY — réponses prêtes

Pourquoi Python et pas C ou Go ?

Watchdog exploite `inotify` nativement — CPU minimal, et portable Linux, Windows, macOS sans recompilation.

Vous gérez les faux positifs comment ?

Les seuils sont configurables. Et surtout : **l'opérateur humain valide ou rejette chaque action avant qu'elle ne s'exécute** — c'est la garantie centrale du système.

Pourquoi pas HTTPS ?

Réseau LAN contrôlé pour ce projet. Le chiffrement HTTPS est la première perspective pour un passage en production.

Quelle différence avec un antivirus ?

Un antivirus cherche des signatures connues. RansomShield analyse le **comportement** : il détecte même une variante jamais vue auparavant.

Testé sur Windows ?

Non — limite assumée dans ce travail. Linux uniquement. C'est une perspective d'extension explicite.

La solution passe-t-elle à l'échelle ?

Testé sur 3 VMs. L'API Laravel peut monter en charge, mais des tests de performance à grande échelle restent à conduire.

Reformuler la question avant de répondre. Garder les captures d'écran prêtes.